1. What is a Function?

A **function** is a block of code that performs a **specific task** and can be reused whenever needed.

Why Use Functions?

1. Code reuse
2. Easier to manage and debug
3. Keeps code clean and organized
4. Reduces repetition

Basic Function Structure

Python

```
def function_name():
    # code
```

C++

```
return_type function_name() {
    // code
}
```

Java

```
return_type functionName() {
    // code
}
```

Types of Functions

Type	Example
No arguments, no return	`printHello()`
Arguments, no return	`printSum(a, b)`
Arguments with return value	`int add(int a, int b)`
No arguments, returns value	`int getInput()`

Practice Questions Using Functions

Question 1: Add Two Integers

LeetCode #2235

Python

```
def sum(num1, num2):
    return num1 + num2
```

C++

```
int sum(int num1, int num2) {
    return num1 + num2;
}
```

Java

```
public int sum(int num1, int num2) {
    return num1 + num2;
}
```

Question 2: Subtract Product and Sum of Digits

LeetCode #1281

Python

```
def subtractProductAndSum(n):
    s = 0
    p = 1
    while n:
        digit = n % 10
        s += digit
        p *= digit
        n //= 10
    return p - s
```

C++

```
int subtractProductAndSum(int n) {
    int s = 0, p = 1;
    while (n) {
        int d = n % 10;
```

```
        s += d;
        p *= d;
        n /= 10;
    }
    return p - s;
}
```

Java

```
public int subtractProductAndSum(int n) {
    int sum = 0, product = 1;
    while (n > 0) {
        int digit = n % 10;
        sum += digit;
        product *= digit;
        n /= 10;
    }
    return product - sum;
}
```

Question 3: Number of Steps to Reduce Number to Zero

LeetCode #1342

Python

```
def numberOfSteps(num):
    steps = 0
    while num:
        if num % 2 == 0:
            num //= 2
        else:
            num -= 1
        steps += 1
    return steps
```

C++

```
int numberOfSteps(int num) {
    int steps = 0;
    while (num) {
        if (num % 2 == 0)
            num /= 2;
        else
            num -= 1;
        steps++;
    }
    return steps;
}
```

Java

```
public int numberOfSteps(int num) {
    int steps = 0;
    while (num > 0) {
```

```
        if (num % 2 == 0)
            num /= 2;
        else
            num -= 1;
        steps++;
    }
    return steps;
}
```

What is Recursion?

Recursion happens when a **function calls itself** to solve smaller versions of the same problem.

Example

Factorial:

```
5! = 5 × 4 × 3 × 2 × 1
    = 5 × factorial(4)
```

Recursive Function Structure

```
function recursiveFunction(parameters):
    if base condition:
        return result
    else:
        return recursiveFunction(smaller_input)
```

Important Terms

1. **Base Case** → Stops recursion.
2. **Recursive Case** → Function calls itself.

Structure in Python

```
def func(n):
    if base_case:
        return result
    else:
        return func(n - 1)
```

Structure in C++ / Java

```
return_type func(type n) {
    if (base_case)
        return result;
    else
        return func(n - 1);
}
```

☑ Print Numbers from 1 to N using Recursion

C++ Code

```
#include <iostream>
using namespace std;

void print1ToN(int n) {
    if (n == 0) return;

    print1ToN(n - 1);
    cout << n << " ";
}

int main() {
    print1ToN(5);
    return 0;
}
```

Java Code

```
public class Main {

    static void print1ToN(int n) {
        if (n == 0) return;

        print1ToN(n - 1);
        System.out.print(n + " ");
    }

    public static void main(String[] args) {
        print1ToN(5);
    }
}
```

Python Code

```
def print1ToN(n):
    if n == 0:
        return
    print1ToN(n - 1)
    print(n, end=" ")

print1ToN(5)
```

☑ Print Numbers from N to 1 using Recursion

C++ Code

```
#include <iostream>
using namespace std;

void printNTo1(int n) {
    if (n == 0) return;

    cout << n << " ";
    printNTo1(n - 1);
}

int main() {
    printNTo1(5);
    return 0;
}
```

Java Code

```
public class Main {

    static void printNTo1(int n) {
        if (n == 0) return;

        System.out.print(n + " ");
        printNTo1(n - 1);
    }

    public static void main(String[] args) {
        printNTo1(5);
    }
}
```

Python Code

```
def printNTo1(n):
    if n == 0:
        return
    print(n, end=" ")
    printNTo1(n - 1)

printNTo1(5)
```

Example 1: Factorial

Python

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
```

C++

```
#include <iostream>
using namespace std;

int factorial(int n) {
    if (n == 0)
        return 1;
    return n * factorial(n - 1);
}

int main() {
    cout << factorial(5);
    return 0;
}
```